TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN CÔNG NGHỆ THÔNG TIN, ĐIỆN, ĐIỆN TỬ

Chương 3 (part1):
Phát hiện đặc trưng và đối sánh ảnh

TS. Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

Phát hiện đặc trưng và đối sánh ảnh

Phát hiện đặc trưng và đối sánh ảnh là hai khái niệm cốt lõi trong lĩnh vực thị giác máy tính, đặc biệt trong các ứng dụng như nhận dạng hình ảnh, tìm kiếm hình ảnh, lập bản đồ 3D và nhiều hơn nữa.

Đặc trưng (feature) là một phần thông tin có liên quan để giải quyết công việc tính toán liên quan đến một ứng dụng nhất định. Các đặc trưng có thể là các cấu trúc cụ thể trong ảnh như các điểm, các cạnh hoặc các đối tượng. Đặc trưng cũng có thể là kết quả của hoạt động vùng lân cận hoặc phát hiện đặc trưng thực hiện trên hình ảnh.

Có **2 loại đặc trưng chính** như sau:

- Các đặc trưng nằm ở vị trí cụ thể trong hình ảnh, chẳng hạn như đỉnh núi, góc tòa nhà, ô cửa hoặc các mảng tuyết có hình dạng thú vị. Các đặc trưng được khoanh vùng này thường được gọi là các ***đặc trưng chính*** (keypoint features hoặc corners) và thường được mô tả bằng sự xuất hiện của các mảng pixel xung quanh vị trí điểm.
- Các đặc trưng có thể được đối sánh dựa trên hướng và diện mạo cục bộ của chúng (biên dạng cạnh – ***edge profiles***) được gọi là edges và chúng cũng có thể là chỉ báo tốt về ranh giới đối tượng và các sự kiện khớp trong chuỗi hình ảnh.

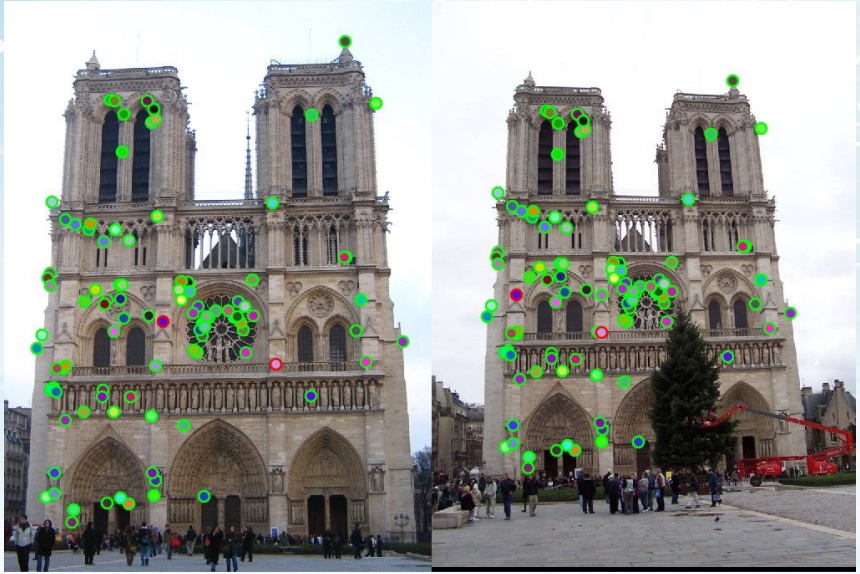
Thành phần chính của phát hiện và đối sánh đặc trưng:

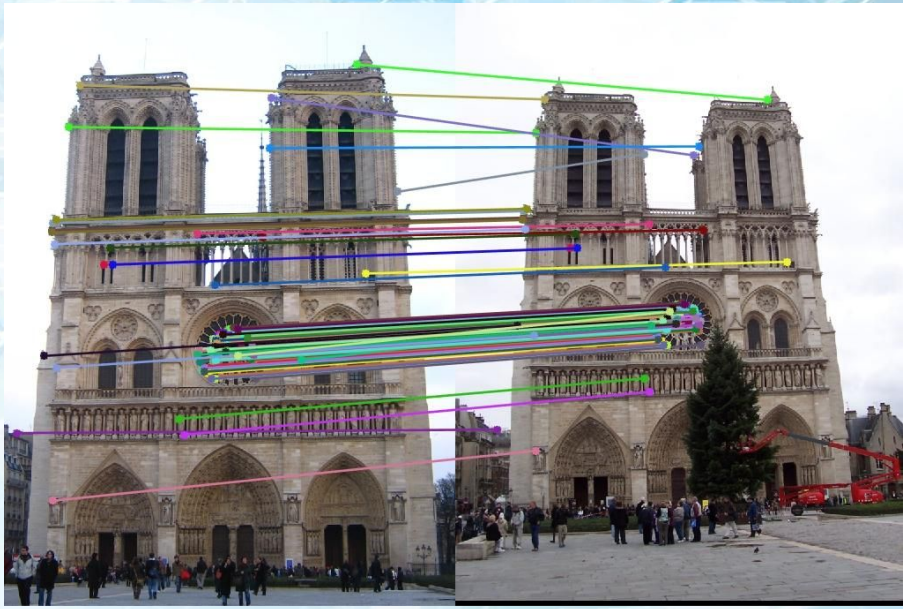
- Phát hiện: Xác định Điểm quan tâm (***Interest Point***)
- Mô tả: Hình thức cục bộ xung quanh mỗi điểm đặc trưng được mô tả theo một cách nào đó (lý tưởng là) bất biến dưới những thay đổi về độ chiếu sáng, dịch chuyển, tỷ lệ và độ xoay trong mặt phẳng. Ta thường kết thúc bằng một vectơ mô tả cho mỗi điểm đặc trưng (feature vector).
- Đối sánh: Các bộ mô tả được so sánh trên các hình ảnh để xác định các đặc điểm tương tự. Đối với hai hình ảnh,ta có thể nhận được một tập hợp các cặp (Xi, Yi) ↔ (Xi`, Yi`), trong đó (Xi, Yi) là một đặc trưng trong một ảnh và (Xi`, Yi`) đặc trưng của nó trong ảnh kia hình ảnh.

Phát hiện đặc trưng và đối sánh ảnh

Ứng dụng của phát hiện đặc trưng và đối sánh ảnh:

- **Nhận dạng vật thể:** Xác định các vật thể trong hình ảnh bằng cách so sánh chúng với các mẫu đã biết.
- **Tìm kiếm hình ảnh:** Tìm các hình ảnh tương tự trong một cơ sở dữ liệu lớn.
- **Lập bản đồ 3D:** Tạo các bản đồ 3D chi tiết từ nhiều hình ảnh.
- **Theo dõi vật thể:** Theo dõi chuyển động của các vật thể trong video.
- **Khôi phục hình ảnh:** Khôi phục các hình ảnh bị hư hỏng.





Phát hiện đặc trưng và đối sánh ảnh

Phát hiện đặc trưng (Feature Detection) là quá trình xác định các điểm đặc biệt trong một hình ảnh (còn gọi là điểm quan tâm hay điểm đặc trưng), những điểm này mang đặc tính ổn định và có thể phân biệt được với các điểm khác, ngay cả khi hình ảnh bị biến đổi bởi các yếu tố như thay đổi góc nhìn, ánh sáng, hoặc nhiễu.
Các đặc trưng này đóng vai trò giúp máy tính hiểu và so sánh các hình ảnh khác nhau.

Các thuật toán phổ biến:
- SIFT (Scale-Invariant Feature Transform)
- SURF (Speeded Up Robust Features)
- ORB (Oriented FAST and Rotated BRIEF)

Các đặc trưng thường tìm kiếm:
- Điểm góc: Những điểm có sự thay đổi đột ngột về cường độ ánh sáng theo nhiều hướng.
- Vùng có kết cấu: Những vùng có các mẫu lặp đi lặp lại.
- Đường biên: Những đường phân cách giữa các vùng có cường độ ánh sáng khác nhau.

Đối sánh ảnh (Image Matching) - là quá trình so sánh hai hoặc nhiều hình ảnh để tìm ra các vùng tương đồng.

Quá trình:
- **Phát hiện đặc trưng:** Tìm các điểm đặc trưng trong cả hai hình ảnh.
- **Mô tả đặc trưng:** Tính toán các mô tả đặc trưng (descriptor) cho mỗi điểm, thường là các vector số.
- **Đối sánh đặc trưng:** So sánh các mô tả đặc trưng để tìm các cặp điểm tương ứng giữa hai hình ảnh.
- **Ước lượng biến đổi:** Sử dụng các cặp điểm tương ứng để ước lượng các biến đổi hình học giữa hai hình ảnh (ví dụ: phép tịnh tiến, xoay, tỉ lệ).

Các phương pháp:
- Đối sánh dựa trên đặc trưng cục bộ
- Đối sánh dựa trên mô hình

SIFT (Scale-Invariant Feature Transform)

SIFT là một thuật toán trong lĩnh vực xử lý ảnh, được sử dụng để tìm kiếm và khớp các điểm đặc trưng (***feature points***) trong một hình ảnh, bất kể hình ảnh đó bị biến đổi như thế nào về kích thước, xoay, hoặc thay đổi độ sáng. Các điểm đặc trưng này giống như những dấu vân tay độc nhất của một hình ảnh, giúp cho việc so sánh, nhận dạng hình ảnh trở nên chính xác hơn.

Ưu điểm của SIFT

- Độ bền cao: SIFT có khả năng phát hiện các đặc trưng rất ổn định, ngay cả khi hình ảnh bị nhiễu hoặc biến dạng.
- Tính phân biệt cao: Các mô tả SIFT rất đặc trưng, giúp dễ dàng phân biệt các vật thể khác nhau.
- Khả năng mở rộng: SIFT có thể được áp dụng cho các hình ảnh có độ phân giải cao và các cảnh phức tạp.

Hạn chế của SIFT

- Tính toán tốn kém: Việc tính toán các đặc trưng SIFT có thể rất tốn kém về mặt tính toán, đặc biệt là đối với các hình ảnh lớn.
- Độ nhạy cảm với nhiễu: Mặc dù SIFT có độ bền cao, nhưng nó vẫn có thể bị ảnh hưởng bởi một số loại nhiễu, chẳng hạn như nhiễu Gaussian.

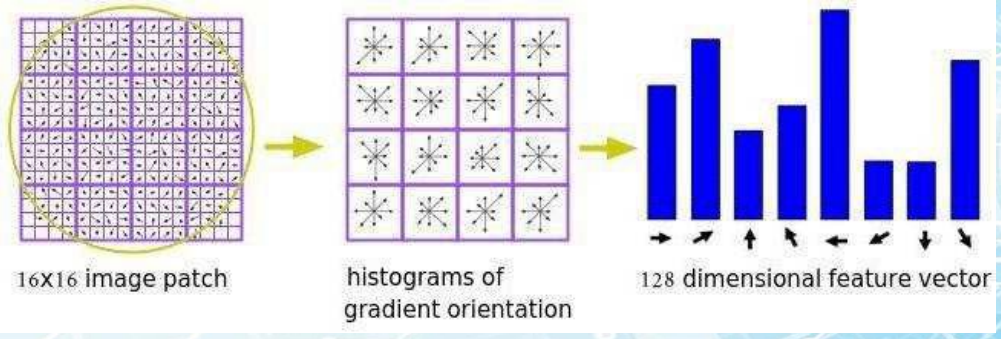
Quy trình hoạt động của SIFT

1. **Xác định điểm cực trị:** Tìm các điểm trong ảnh có giá trị gradient lớn nhất so với các điểm lân cận.
2. **Tạo mô tả tử:** Tính toán một vector đặc trưng cho mỗi điểm cực trị, mô tả các thông tin về gradient và hướng của các điểm xung quanh.
3. **Loại bỏ các điểm kém ổn định:** Loại bỏ các điểm đặc trưng không đủ ổn định để đảm bảo tính chính xác của kết quả.
4. **Khớp các điểm đặc trưng:** So sánh các vector đặc trưng của hai hình ảnh để tìm các cặp điểm tương ứng.

Gradient hình ảnh trong SIFT

Gradient hình ảnh là một khái niệm quan trọng trong thuật toán SIFT (Scale-Invariant Feature Transform), đóng vai trò cốt lõi trong việc phát hiện và mô tả các đặc trưng bất biến trong hình ảnh. Gradient cho ta biết hình ảnh thay đổi như thế nào từ điểm này sang điểm khác. Gradient thường được biểu diễn dưới dạng một vector, bao gồm độ lớn và hướng:

- Độ lớn gradient: Cho biết tốc độ thay đổi cường độ sáng tại một điểm. Các điểm có độ lớn gradient cao thường là các cạnh, góc hoặc các vùng có kết cấu trong hình ảnh.
- Hướng gradient: Cho biết hướng của sự thay đổi cường độ sáng nhanh nhất.



Vai trò của gradient trong SIFT

- **Phát hiện điểm cực trị:** SIFT tìm kiếm các điểm có độ lớn gradient lớn và các điểm mà độ lớn gradient đạt cực đại hoặc cực tiểu theo các hướng khác nhau. Những điểm này thường là các điểm đặc trưng ổn định của hình ảnh.
- **Tính toán mô tả** đặc trưng cho mỗi điểm. Mô tả này dựa trên histogram của các hướng gradient trong một vùng lân cận của điểm đặc trưng. Histogram này không chỉ chứa thông tin về độ lớn gradient mà còn cả thông tin về phân bố hướng của gradient, giúp mô tả đặc trưng trở nên bất biến hơn đối với các phép biến đổi hình học như xoay.
- **Tạo ra các đặc trưng bất biến** đối với các phép biến đổi tỷ lệ, xoay và một phần đối với các thay đổi về ánh sáng. Điều này làm cho các đặc trưng SIFT trở nên rất hữu ích trong việc so sánh và khớp nối các hình ảnh.

6

SIFT (Scale-Invariant Feature Transform)

Các Hàm Gradient Thường Sử Dụng trong SIFT

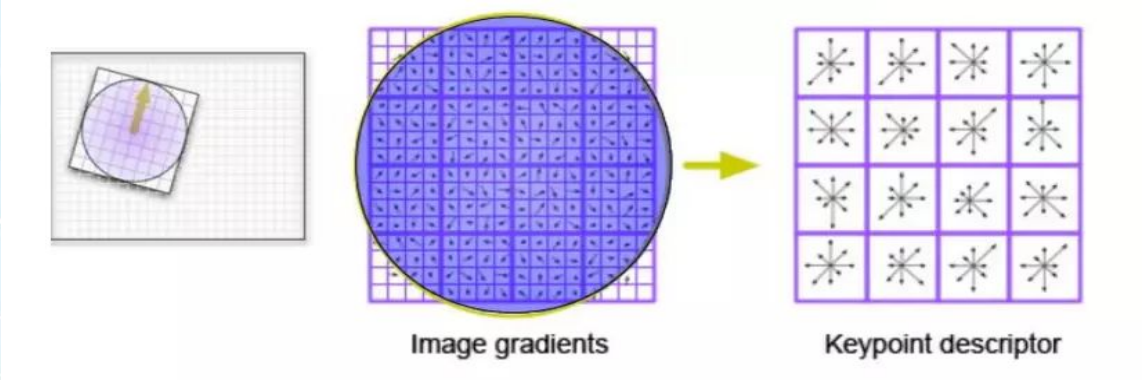
Mặc dù không có một hàm gradient cụ thể nào được định nghĩa riêng cho SIFT, nhưng các hàm gradient cơ bản sau đây thường được sử dụng:

- **Gradient xấp xỉ:**
  - **Sobel:** Đây là một bộ lọc thường được sử dụng để tính toán gradient xấp xỉ. Sobel sử dụng các ma trận convolution để tính toán đạo hàm theo phương ngang và phương dọc.
  - **Prewitt:** Tương tự như Sobel, Prewitt cũng là một bộ lọc để tính toán gradient xấp xỉ.
  - **Scharr:** Là một phiên bản cải tiến của Sobel, cho kết quả chính xác hơn với ít nhiễu hơn.
- **Gradient chính xác:**
  - **Sử dụng đạo hàm số:** Trong một số trường hợp, có thể tính toán chính xác gradient bằng cách sử dụng các công thức đạo hàm của hàm mô tả hình ảnh. Tuy nhiên, phương pháp này thường tốn kém về mặt tính toán hơn.

Quá trình Tính Toán Gradient trong SIFT

1. **Lọc Gaussian:** Hình ảnh đầu vào được lọc Gaussian để giảm nhiễu.
2. **Tính toán hiệu của Gaussian (Difference of Gaussians - DoG):** Tính hiệu giữa hai ảnh Gaussian có các tham số khác nhau để tìm các điểm cực trị.
3. **Lọc các điểm cực trị:** Loại bỏ các điểm cực trị yếu và các điểm nằm trên các cạnh.
4. **Gán hướng:** Tính toán histogram của các hướng gradient trong một vùng lân cận của mỗi điểm cực trị để gán hướng cho điểm đó.
5. **Tạo mô tả:** Tính toán histogram của các hướng gradient trong các vùng con của một vùng lân cận lớn hơn để tạo ra mô tả đặc trưng.

SIFT (Scale-Invariant Feature Transform)



1. Với đầu vào là một hình ảnh >>> SIFT >>> các keypoint. Mỗi đối tượng trong hình ảnh sẽ cho ra rất nhiều các keypoint khác nhau, để ta phân biệt được các keypoint này với nhau sẽ thông qua một vector 128 chiều hay còn gọi là descriptor. Các descriptor này sẽ được dùng để nhận dạng đối tượng trong ảnh, hay dùng cho các bài toán classification.
2. Hình ảnh sau khi áp dụng biến đổi SIFT, ứng với mỗi keypoint ta sẽ thu được: toạ độ keypoint, scale và orientation của keypoint, descriptor. Các mũi tên trong hình dưới vẽ nhờ vào scale và orientation.
3. 4x4 Gradient window
4. Histogram of 4x4 samples per window in 8 directions
5. Gaussian weighting around center
6. 4x4x8 = 128 dimensional feature vector

SURF (Speeded Up Robust Features)

SURF (Speeded Up Robust Features) là một thuật toán trong lĩnh vực thị giác máy tính, được thiết kế để phát hiện và mô tả các đặc trưng trong hình ảnh một cách nhanh chóng và ổn định. SURF được xem như là một phiên bản cải tiến của SIFT với mục tiêu tăng tốc độ tính toán mà vẫn đảm bảo độ chính xác.

Các cải tiến chính của SURF so với SIFT

- Bộ lọc Box: SURF sử dụng bộ lọc Box để tính toán Hessian matrix một cách hiệu quả hơn so với việc sử dụng hiệu của Gaussian (DoG) trong SIFT.
- Mô tả đặc trưng: SURF sử dụng mô tả đặc trưng dựa trên Hessian matrix, đơn giản hơn và nhanh hơn so với histogram của các hướng gradient trong SIFT.
- Inteval: SURF sử dụng một cấu trúc dữ liệu gọi là Interval để tăng tốc độ tìm kiếm các điểm cực trị

Tính năng	SIFT	SURF
Tốc độ	Tương đối chậm	Rất nhanh
Độ chính xác	Cao	Cao, gần tương đương SIFT
Bất biến	Tỷ lệ, xoay, một phần ánh sáng	Tỷ lệ, xoay, một phần ánh sáng
Mô tả đặc trưng	Histogram của các hướng gradient	Hessian matrix

SURF (Speeded Up Robust Features)

Các bước chính của thuật toán SURF

- Xác định điểm đặc trưng:
  Tính toán Hessian matrix tại mỗi điểm pixel.
  Tìm các điểm cực đại và cực tiểu của Hessian matrix.
- Gán hướng:
  Tính toán histogram của các hướng gradient trong một vùng lân cận của điểm đặc trưng.
  Xác định hướng chính của điểm đặc trưng.
- Tạo mô tả đặc trưng:
  Tính toán Hessian matrix trong các vùng con của một vùng lân cận lớn hơn.
  Tạo một vector mô tả đặc trưng dựa trên các giá trị của Hessian matrix.

Ưu điểm của SURF

- Tốc độ cao: SURF nhanh hơn SIFT nhiều lần, đặc biệt khi xử lý các hình ảnh lớn.
- Độ chính xác cao: SURF có khả năng phát hiện và mô tả các đặc trưng một cách chính xác, tương đương với SIFT.
- Tính ổn định: SURF có tính bất biến cao đối với các phép biến đổi hình học như thay đổi tỷ lệ, xoay và một phần đối với các thay đổi về ánh sáng.

Nhược điểm của SURF

- Ít được sử dụng rộng rãi: So với SIFT, SURF ít được sử dụng rộng rãi hơn trong các nghiên cứu gần đây.
- Tính độc quyền: Một số phần của thuật toán SURF có thể bị ràng buộc bởi các bằng sáng chế.

Ứng dụng của SURF

- Nhận dạng vật thể: So sánh các đặc trưng SURF giữa các hình ảnh để xác định xem các vật thể có giống nhau hay không.
- Theo dõi vật thể: Theo dõi chuyển động của các vật thể trong các video bằng cách so sánh các đặc trưng SURF giữa các khung hình liên tiếp.
- Ghép nối hình ảnh: Kết hợp nhiều hình ảnh để tạo thành một hình ảnh toàn cảnh.
- Tái tạo 3D: Tạo ra các mô hình 3D từ các hình ảnh 2D.

ORB (Oriented FAST and Rotated BRIEF)

ORB (Oriented FAST và Rotated BRIEF) là một thuật toán trong lĩnh vực thị giác máy tính được thiết kế để phát hiện và mô tả các đặc trưng trong hình ảnh một cách nhanh chóng và ổn định. Nó được xem như là một phiên bản cải tiến của SIFT (Scale-Invariant Feature Transform), với mục tiêu tăng tốc độ tính toán mà vẫn đảm bảo độ chính xác.

Các thành phần chính của ORB

- **FAST (Features from Accelerated Segment Test)**: Một bộ phát hiện góc, xác định các điểm đặc trưng tiềm năng bằng cách so sánh cường độ của một pixel với các pixel xung quanh. Đây là một phương pháp nhanh và hiệu quả để phát hiện các điểm đặc trưng.
- **Oriented FAST:** Thêm thông tin về hướng cho mỗi điểm đặc trưng bằng cách tính toán hướng của gradient cường độ trong một vùng lân cận của điểm đặc trưng. Thông tin về hướng này giúp các điểm đặc trưng trở nên ổn định hơn với phép quay.
- **Rotated BRIEF:**

Một mô tả nhị phân mô tả sự xuất hiện của một điểm đặc trưng.

Nó chọn một tập hợp các phép kiểm tra nhị phân và tính toán một chuỗi nhị phân cho mỗi điểm đặc trưng, dựa trên so sánh cường độ giữa các cặp pixel.

Rotated BRIEF cải thiện độ ổn định của mô tả đối với phép quay bằng cách xoay mô hình các phép kiểm tra nhị phân theo hướng của điểm đặc trưng.

Ưu điểm của ORB

- Tốc độ cao: ORB nhanh hơn SIFT và SURF nhiều lần, phù hợp cho các ứng dụng thời gian thực.
- Độ chính xác cao: ORB cung cấp độ chính xác tốt về việc phát hiện và mô tả các điểm đặc trưng.
- Không có bằng sáng chế: ORB không bị ràng buộc bởi bất kỳ bằng sáng chế nào, do đó có thể sử dụng miễn phí cho mục đích thương mại.

Nhược điểm của ORB

- Ít ổn định hơn với các thay đổi quan điểm lớn: ORB có thể không ổn định bằng SIFT hoặc SURF đối với các thay đổi quan điểm lớn.
- Khả năng phân biệt kém hơn: Các mô tả nhị phân được sử dụng bởi ORB có thể kém khả năng phân biệt hơn so với các mô tả phức tạp hơn được sử dụng bởi SIFT và SURF.

So sánh SIFT, SURF và ORB

	SIFT	SUFT	ORB
Cách thức hoạt động	- Xác định các điểm cực trị trong không gian quy mô (scale-space). - Gán hướng cho các điểm đặc trưng dựa trên gradient. - Tạo mô tả đặc trưng dựa trên histogram của các hướng gradient.	- Sử dụng bộ lọc Box để tính toán Hessian matrix hiệu quả hơn. - Mô tả đặc trưng dựa trên Hessian matrix.	- Sử dụng FAST để phát hiện các điểm đặc trưng. - Gán hướng cho các điểm đặc trưng. - Sử dụng BRIEF để tạo mô tả đặc trưng nhị phân.
Ưu điểm	Độ chính xác cao, bất biến tốt với các phép biến đổi hình học (tỷ lệ, xoay, một phần ánh sáng).	Tốc độ nhanh hơn SIFT đáng kể, vẫn giữ được độ chính xác cao.	Tốc độ cực nhanh, không có bằng sáng chế, độ chính xác chấp nhận được.
Nhược điểm	Tốc độ chậm, phức tạp về mặt tính toán.	Ít bất biến với các phép biến đổi affine so với SIFT.	Độ chính xác có thể thấp hơn SIFT và SURF trong một số trường hợp, ít bất biến với các phép biến đổi affine.

Đối sánh dựa trên đặc trưng cục bộ (Local feature-based matching)

Nguyên lý: Phát hiện các điểm đặc trưng nổi bật trong hình ảnh, sau đó so sánh các mô tả của các điểm đặc trưng này giữa các hình ảnh để tìm ra các điểm tương ứng.

Ưu điểm:

- Khả năng xử lý các biến đổi hình học, ánh sáng và nhiễu tốt.
- Linh hoạt, có thể áp dụng cho nhiều loại hình ảnh khác nhau.

Nhược điểm:

- Có thể tốn nhiều thời gian tính toán, đặc biệt đối với các hình ảnh có độ phân giải cao.
- Độ chính xác phụ thuộc vào chất lượng của các điểm đặc trưng được phát hiện.

Các thuật toán điển hình: SIFT, SURF, ORB, KAZE, AKAZE.

Quy trình:

- **Phát hiện điểm đặc trưng:** Tìm các điểm trong hình ảnh có độ tương phản cao và ổn định với các biến đổi hình học.
- **Tính toán mô tả:** Tạo một vector đặc trưng mô tả xung quanh mỗi điểm đặc trưng, bao gồm thông tin về cường độ, gradient, v.v.
- **So sánh mô tả:** So sánh các vector đặc trưng giữa hai hình ảnh để tìm các cặp điểm tương ứng.

Đối sánh dựa trên mô hình (Model-based matching)

Nguyên lý: Sử dụng các mô hình toán học để mô tả các đối tượng trong hình ảnh và so sánh các mô hình này để tìm ra sự tương đồng.

Ưu điểm:

- Khả năng xử lý các biến dạng phức tạp của đối tượng.
- Có thể tìm kiếm các đối tượng cụ thể theo một mô hình đã biết

Nhược điểm:

- Cần một lượng lớn dữ liệu huấn luyện để xây dựng mô hình.
- Nhạy cảm với sự thay đổi trong hình dạng và kết cấu của đối tượng.

Các thuật toán điển hình:

- Active Shape Models (ASM),
- Active Appearance Models (AAM),
- các mô hình dựa trên học sâu (deep learning).

Quy trình:

- Xây dựng mô hình: Tạo một mô hình toán học để biểu diễn hình dạng và kết cấu của đối tượng.
- Tìm kiếm và khớp: Tìm kiếm các điểm đánh dấu trong hình ảnh và điều chỉnh mô hình sao cho phù hợp nhất với dữ liệu hình ảnh.

Đối sánh hình ảnh (Image Matching)

Việc lựa chọn phương pháp đối sánh hình ảnh phụ thuộc vào nhiều yếu tố, bao gồm:

- **Tính chất của dữ liệu:** Độ phức tạp của hình ảnh, mức độ nhiễu, biến đổi hình học.
- **Yêu cầu về độ chính xác và tốc độ:** Mức độ chính xác cần đạt được và thời gian thực thi.
- **Kiến thức trước về đối tượng:** Có hay không có thông tin về hình dạng, kết cấu của đối tượng.

Đối sánh dựa trên đặc trưng cục bộ thích hợp cho các bài toán tổng quát, không yêu cầu kiến thức trước về đối tượng.
Đối sánh dựa trên mô hình thích hợp cho các bài toán đòi hỏi độ chính xác cao, cần tìm kiếm các đối tượng cụ thể theo một mô hình đã biết.

Ví dụ:

- **Đối sánh dựa trên đặc trưng cục bộ:** Tìm kiếm các điểm tương đồng giữa hai bức ảnh để tạo ra một bức ảnh toàn cảnh.
- **Đối sánh dựa trên mô hình:** Nhận dạng khuôn mặt trong một video giám sát.

Độ chính xác của quá trình đối sánh ảnh phụ thuộc vào nhiều yếu tố, từ chất lượng hình ảnh đến thuật toán được sử dụng.
Một số yếu tố chính ảnh hưởng đến độ chính xác này:

1. Chất lượng hình ảnh: độ phân giải, độ nhiễu, ánh sáng, biến dạng

2. Thuật toán đối sánh:

- **Loại thuật toán.** Mỗi thuật toán có những ưu nhược điểm khác nhau. Ví dụ, SIFT rất chính xác nhưng chậm, trong khi ORB nhanh hơn nhưng có thể kém chính xác hơn trong một số trường hợp.
- **Các tham số của thuật toán.** Việc thiết lập các tham số của thuật toán một cách phù hợp rất quan trọng. Ví dụ, số lượng điểm đặc trưng, ngưỡng tương đồng, v.v.
- **Phương pháp loại bỏ điểm ngoại lai.** Các điểm ngoại lai (outliers) có thể làm giảm độ chính xác của quá trình đối sánh. Việc sử dụng các phương pháp loại bỏ điểm ngoại lai như RANSAC là rất cần thiết.

3. Đặc trưng được sử dụng: loại đặc trưng, mô tả đặc trưng

4. Môi trường làm việc: tốc độ tính toán, bộ nhớ, hệ điều hành

5. Các yếu tố khác: tiền xử lý hình ảnh, phương pháp đánh giá

Bài thực hành 4

So sánh sự tương đồng của các hình ảnh sử dụng Wavelet, Python

I> Mục tiêu bài tập:

- Biết cách sử dụng wavelet biến đổi để trích xuất thông tin cụ thể và so sánh sự tương thích giữa các hình ảnh.
- Làm quen với PyWavelets thư viện và các công cụ xử lý ảnh trong Python.
- Đánh giá kết quả của hàm băm wavelet phương pháp trong việc xác định các hình ảnh tương thích.

II> Bài toán cụ thể:

1 - Chuẩn bị dữ liệu. Chuẩn bị một tập hợp các hình ảnh, bao gồm các hình ảnh cặp đôi tương tự nhau (ví dụ: cùng một đối tượng với các góc độ khác nhau, các mức độ nhiễu khác nhau) và các hình ảnh cặp không tương tự.

2 - Trích xuất wavelet đặc biệt. Sử dụng wavelet biến đổi để chuyển đổi từng hình ảnh thành một wavelet ma trận.

3 - Tạo mã băm cho mỗi hình ảnh dựa trên các wavelet số đã lượng tử hóa.

4 - So sánh hàm băm. Tính khoảng cách Hamming giữa các mã băm để đánh giá mức độ tương thích giữa các hình ảnh.

5- Đánh giá.

- **Độ chính xác:** Tỷ lệ tính toán của các cặp hình ảnh được phân loại đúng (tương tự/không tương tự).
- **Độ nhạy:** Tỷ lệ các cặp hình ảnh tương thích được phân loại đúng.
- **Độ đặc biệt:** Tỷ lệ tính toán của các cặp hình ảnh không tương thích với loại phân tích đúng.
- **Đường cong ROC:** Vẽ đường cong ROC để đánh giá hiệu suất của thuật toán.

III> Bài tập nâng cao:

1 - Thực hiện khảo sát về các phương pháp băm wavelet khác nhau và so sánh hiệu suất của chúng.

2 - Xây dựng ứng dụng tìm kiếm hình ảnh dựa trên hàm băm wavelet.

Bài thực hành 4

So sánh sự tương đồng của các hình ảnh sử dụng Wavelet, Python

IV> Tham khảo:

1) Hash wavelet (Băm hình ảnh wavelet)

Băm hình ảnh wavelet là một kỹ thuật được sử dụng để tạo mã băm nhỏ gọn cho hình ảnh, có thể được sử dụng để tìm kiếm sự tương đồng của hình ảnh hoặc phát hiện vi phạm bản quyền. Quá trình này bao gồm:

1. **Biến đổi wavelet:** Phân tích hình ảnh thành các thành phần tần số.
2. **Lượng tử hóa hệ số:** Giảm độ chính xác của các hệ số.
3. **Tạo băm:** Tạo mã băm dựa trên các hệ số lượng tử.

V> Triển khai với Python và PyWavelets

```python
import pywt
import numpy as np

def wavelet_hash(image_path, wavelet='db4', level=3):
  """
  Calculates the wavelet hash of an image.

  Args:
    image_path: Path to the image file.
    wavelet: Wavelet type to use.
    level: Decomposition level.

  Returns:
    A list of bits representing the hash code.
  """

  # Load the image
  img = plt.imread(image_path)

  # Perform wavelet transform
  coeffs = pywt.wavedec2(img, wavelet, level=level)

  # Quantize coefficients
  coeffs = [pywt.quantize(c, 2) for c in coeffs]

  # Flatten the quantized coefficients
  flattened = np.concatenate([c.flatten() for c in coeffs])

  # Generate hash code
  hash_code = []
  for bit in flattened:
    hash_code.append(bit % 2)

  return hash_code

# Example usage:
hash1 = wavelet_hash("image1.jpg")
hash2 = wavelet_hash("image2.jpg")

# Compare hashes
hamming_distance = sum(abs(a-b) for a,b in zip(hash1, hash2))
print("Hamming distance:", hamming_distance)
```

Bài thực hành 4

So sánh sự tương đồng của các hình ảnh sử dụng Wavelet, Python

Khung chương trình xử lý của bài thực hành chương 3

```python
import pywt
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def wavelet_hash(image_path, wavelet='db4', level=1, bits=8):
    # ... (tương tự như phần code đã cung cấp)

# Load dữ liệu
images = ["image1.jpg", "image2.jpg", ...]
hashes = [wavelet_hash(img) for img in images]

# Tính khoảng cách Hamming
def hamming_distance(hash1, hash2):
    # ...

# So sánh và đánh giá
distances = []
for i in range(len(hashes)):
    for j in range(i+1, len(hashes)):
        distance = hamming_distance(hashes[i], hashes[j])
        distances.append(distance)
        # So sánh với nhãn ground truth để tính toán các chỉ số đánh giá

# Vẽ đường cong ROC
# ...

# Đánh giá kết quả
print("Độ chính xác:", accuracy)
print("Độ nhạy:", recall)
print("Độ đặc hiệu:", precision)
```